

Ceritage Jewelry ERP Software

End-to-End Build, Database, Payment & Deployment Workflow

A practical engineering blueprint for building the Ceritage Jewelry ERP as an in-house web application: architecture, tech stack, database schema, module workflows, user/admin roles, payment integration, security, and long-term hosting & maintenance.

Prepared as an internal engineering reference | August 2026

0. How to Use This Document

This document turns the Ceritage Jewelry ERP product specification into a concrete, buildable engineering plan. It is organised so a small team (or a solo developer) can go from zero to a live, production system used internally by the jewellery business. Follow the sections in order: architecture → database → module workflows → roles → payments → security → deployment → long-term operation.

1. Project Goal & Scope

Goal: Build a private, internally-hosted web application (not sold to other businesses) that runs the full jewellery retail operation: customers, inventory, billing, purchases, gold exchange, repairs, karigar (artisan) work, accounting, and reporting — accessible to staff from the shop counter and to the owner/admin from anywhere.

- **Single business, multiple roles** — no multi-tenant/licensing complexity needed since it's for internal use only.
- **Web-first**, so it works on desktop billing counters, tablets, and phones without separate installs.
- **Offline-tolerant billing** — counter must keep working during brief internet drops.
- **Bank-grade security** for customer KYC, GST data, and payment records.
- **Room to grow** into multi-branch later without re-architecting.

2. Recommended Technology Stack

This keeps the spec's original choices (React / Node.js / PostgreSQL) but fills in the supporting pieces needed for a real production system.

Layer	Technology	Why
Frontend (Web)	React + TypeScript, Vite, TailwindCSS, shadcn/ui, React Query	Fast billing UI, works well on touch counters/tablets
State/Forms	React Query (server state) + Zustand (UI state) + React Hook Form + Zod	Predictable data flow, strong validation for GST/KYC forms
Backend API	Node.js + TypeScript, NestJS (or Express + Fastify)	Structured, modular backend that matches the many modules in the spec
Database	PostgreSQL 16	ACID compliant — critical for ledgers, invoices, stock counts
ORM	Prisma or TypeORM	Type-safe queries, migrations, schema versioning
Cache/Queue	Redis	Session cache, rate-limiting, background jobs (SMS/WhatsApp, reports)
File/Image Storage	S3-compatible object storage (AWS S3 / Cloudflare R2 / MinIO for self-host)	Product images, hallmark certificates, KYC docs
Auth	JWT access + refresh tokens, bcrypt/argon2 password hashing, optional TOTP 2FA	Role-based multi-user login as required by spec
Realtime Sync	WebSockets (Socket.IO) or Server-Sent Events	Live stock updates across branches/counters
PDF/Invoice Generation	Puppeteer or pdf-lib on the backend	GST invoices, e-way bills, certificates

Layer	Technology	Why
Barcode/QR	bwip-js / qrcode (generation), html5-qrcode or a USB/Bluetooth scanner (backend just reads scanner input as keyboard text)	Product tagging & billing scan-in
Desktop wrapper (optional)	Electron shell loading the same web app	Matches spec's 'Desktop Application' line, offline billing cache
Mobile (optional, later)	React Native or a PWA install of the same React app	Owner dashboard & staff attendance on the go
Payments	Razorpay or PayU (India-focused; UPI/cards/netbanking), Stripe if international	Handles UPI, cards, wallets, EMI mentioned in spec
SMS/WhatsApp	MSG91 / Gupshup / Twilio WhatsApp Business API	Invoice sharing, OTP, birthday/anniversary reminders
Hosting	AWS / DigitalOcean / Hetzner + Docker + Nginx reverse proxy	See Section 9 for full deployment plan
CI/CD	GitHub Actions	Automated test + build + deploy pipeline
Monitoring	Grafana + Prometheus, or hosted (Better Stack / UptimeRobot) + Sentry for errors	Keeps the system reliable for years

Tip: since this is for one business's internal use, you do not need the 'White Label' or multi-tenant licensing modules from the spec — skip those and save significant build time.

3. High-Level System Architecture

A simple, layered architecture is enough — you do not need microservices for a single-business ERP.

3.1 Logical layers

- 1. Client layer** — React web app (counter billing, inventory, admin dashboard), installable as a PWA on tablets/phones; optional Electron wrapper for the main billing PC.
- 2. API layer** — Node.js/NestJS REST (or GraphQL) API, organised into modules that mirror the spec: customers, products, billing, purchase, gold-exchange, repairs, karigar, accounting, reports, auth.
- 3. Domain/service layer** — business logic: GST calculation, tunch/purity conversion, gold-rate locking, stock deduction on sale, ledger posting — kept separate from HTTP controllers so it's testable.
- 4. Data layer** — PostgreSQL (primary data), Redis (cache/queues), S3-compatible storage (files/images).
- 5. Integration layer** — payment gateway, SMS/WhatsApp, live gold-rate API, e-invoice/e-way bill GSP (GST Suvidha Provider) if you enable e-invoicing.
- 6. Background workers** — a queue (BullMQ on Redis) for: sending invoices, daily gold-rate refresh, due-date reminders, nightly backups, report generation.

3.2 One database, modular schema

Use a single PostgreSQL database with clearly separated schemas (or well-prefixed tables) — e.g. *crm*, *inventory*, *billing*, *accounting*, *hr* — instead of separate databases per module. This keeps referential integrity (e.g. an invoice line must point to a real product) enforceable at the database level.

4. Database Design

Below is the core entity list you need before writing any code. Build these as Prisma/TypeORM models with foreign keys and indexes on frequently filtered columns (SKU, phone number, invoice number, date).

4.x Customers (CRM)

Table	Key Fields
customers	id, name, phone, email, address, dob, anniversary, kyc_type, kyc_number, kyc_doc_url, created_at
customer_ledger	id, customer_id, txn_type(debit/credit), amount, ref_type, ref_id, balance_after, created_at
loyalty_points	id, customer_id, points, source, created_at
memberships	id, customer_id, plan_id, start_date, end_date, status

4.x Products & Inventory

Table	Key Fields
products	id, sku, barcode, name, category, sub_category, collection, design_no, gender, hsn_code, gross_weight, net_weight, stone_weight, purity, making_charge_type, making_charge_value, wastage_pct, purchase_price, mrp, hallmark_no, huid_no, branch_id, status
product_images	id, product_id, url, is_primary
stones	id, name, type, color, shape, size, clarity, cut, carat, weight, certificate_no, purchase_price, selling_price
product_stones	product_id, stone_id, qty
stock_ledger	id, product_id, branch_id, txn_type(in/out/transfer/adjust), qty, ref_type, ref_id, created_at
stock_locations	id, branch_id, tray_id, counter_id, product_id, qty
branches	id, name, address, gstin, is_main

4.x Billing & Sales

Table	Key Fields
invoices	id, invoice_no, type(retail/wholesale/estimate/quotation), customer_id, branch_id, subtotal, discount, gst_amount, round_off, grand_total, status, created_by, created_at
invoice_items	id, invoice_id, product_id, qty, rate, making_charge, wastage_amount, gst_pct, line_total
payments	id, invoice_id, mode(cash/upi/card/bank/wallet/emi), amount, gateway_ref, status, created_at
credit_notes / debit_notes	id, invoice_id, reason, amount, created_at
sales_orders	id, customer_id, expected_delivery, advance_paid, status

4.x Purchase & Gold Exchange

Table	Key Fields
suppliers	id, name, phone, gstin, address
purchase_orders / purchase_entries	id, supplier_id, branch_id, total_amount, status, created_at
purchase_items	id, purchase_id, product_id/description, qty, weight, purity, rate, amount
gold_exchange	id, customer_id, metal_type, gross_weight, purity, melting_loss_pct, fine_weight, rate, amount, invoice_id

4.x Karigar (Artisan) & Manufacturing

Table	Key Fields
karigars	id, name, phone, specialization, address
work_orders	id, karigar_id, product_desc, gold_issued_wt, gold_purity, expected_return_date, status
karigar_fine_metal_ledger	id, karigar_id, issued_fine_wt, received_fine_wt, balance_fine_wt
labour_payments	id, karigar_id, work_order_id, amount, paid_on

4.x Accounting

Table	Key Fields
chart_of_accounts	id, name, type(asset/liability/income/expense), parent_id
journal_entries	id, date, narration, created_by
journal_lines	id, journal_id, account_id, debit, credit
vouchers	id, type(payment/receipt/contra/journal), amount, party_id, narration, date

4.x HR & Users

Table	Key Fields
employees	id, name, phone, role_id, branch_id, salary, join_date, status
attendance	id, employee_id, date, check_in, check_out, status
users	id, employee_id, username, password_hash, role_id, is_active, last_login
roles / permissions	id, name; role_permissions(role_id, module, can_view, can_edit, can_delete)
activity_logs	id, user_id, action, module, ref_id, ip_address, created_at

4.x Rates & Config

Table	Key Fields
metal_rates	id, metal(gold/silver/platinum), purity, rate_per_gram, effective_date, source(manual/api)
gst_settings	id, hsn_code, cgst_pct, sgst_pct, igst_pct
app_settings	key, value — company profile, invoice numbering, branding

Indexing rule of thumb: add indexes on every foreign key, on *phone* and *invoice_no* for fast lookup at the billing counter, and on *created_at* for date-range reports. Use PostgreSQL *NUMERIC* (not float) for all money and weight columns.

5. Module-Wise Functional Workflow

Each module below maps directly to the sections in the original specification, described as an operational flow you can hand to a developer as a user story.

5.1 Customer Management

1. Staff searches customer by phone at billing screen; if not found, a quick-add form (name, phone, optional KYC) creates the record inline so billing is never blocked.
2. Full KYC (Aadhaar/PAN, address, photo) can be completed later from the Customer Profile page — required automatically above the government cash-transaction threshold (see Section 5.9).
3. Every invoice, return, and old-gold exchange automatically writes an entry to that customer's ledger and purchase history.
4. A daily scheduled job checks birthdays/anniversaries and queues WhatsApp/SMS greetings.

5.2 Product & Inventory Management

1. New stock is entered via Purchase Entry or direct Product creation, capturing weight, purity, making charges, HUID, and images.
2. On save, the system auto-generates a SKU, barcode, and QR code and lets staff print adhesive labels.
3. Stock quantity lives in ``stock_ledger`` as a running total per branch/tray/counter — every sale, transfer, damage report, or audit writes a ledger row rather than editing a single 'quantity' field, giving a full audit trail.
4. Low-stock and reorder-level alerts run as a nightly job and show on the admin dashboard.

5.3 Billing (POS)

1. Cashier scans/searches product → item added to cart with live weight, purity, making charge, and today's metal rate auto-filled from ``metal_rates``.
2. System computes: (metal value + making charge + stone value) – discount, then applies CGST/SGST or IGST from ``gst_settings`` by HSN code.
3. Customer selects payment mode(s) — supports split payment (e.g. part cash + part UPI) written as multiple rows in ``payments``.
4. On confirming payment, the system: (a) deducts stock via ``stock_ledger``, (b) posts accounting entries, (c) generates a PDF invoice, (d) sends it by SMS/WhatsApp/Email.
5. If internet drops mid-sale, the Electron/PWA client queues the invoice locally and syncs once connectivity returns (see Section 3.1 background workers).

5.4 Purchase Management

1. Create a Purchase Order to a supplier → on goods arrival, staff records a Goods Received Note matching PO items.
2. Old-gold/old-silver purchase from walk-in customers follows the same flow but posts to ``gold_exchange`` and updates the customer ledger instead of a supplier ledger.
3. Supplier payments and returns update the supplier ledger the same way customer payments update the customer ledger.

5.5 Gold Exchange

1. Weigh the old item → apply melting-loss % and purity conversion to compute fine (24K-equivalent) weight.
2. Value is calculated at today's rate and either paid out or adjusted against a new purchase invoice in the same transaction.

5.6 Repair Job Card

1. Create a Job Card with item description, photos, promised charge, and delivery date; system auto-generates a token/receipt for the customer.
2. Status moves through a fixed pipeline: Received → With Karigar → Ready → Delivered, with SMS/WhatsApp notification on each status change.

5.7 Order Booking & Gold-Rate Locking

1. Customer pays an advance for a custom/bridal order; staff chooses Fixed-Rate (locked today) or Floating-Rate (rate on delivery day) booking.
2. On final delivery, the system auto-adjusts the invoice: for fixed-rate it uses the locked rate for the booked weight; for floating it re-prices at the current rate, then settles the advance against the final bill.

5.8 Karigar (Artisan) Management

1. Issue raw gold/silver + stones to a karigar against a Work Order, recording issued fine weight.
2. On return, record received weight/purity; the system computes wastage automatically and updates the karigar's fine-metal balance and labour-charge payable.

5.9 Accounting & Compliance

1. Every billing, purchase, exchange, and payment transaction automatically creates double-entry journal lines against the Chart of Accounts — staff never touches raw accounting screens.
2. Cash sales above the statutory threshold trigger a mandatory PAN/Aadhaar or Form 60/61 capture prompt and log a TCS entry (see Part 2 of the spec, 'High-Value Compliance').
3. Month-end, the system rolls journal lines into Trial Balance, P&L, and Balance Sheet reports automatically.

5.10 Reports & Analytics Dashboard

1. Pre-built report queries (daily/monthly/yearly sales, stock, GST, profit) run against read replicas or materialized views so they don't slow down live billing.
2. The Admin Dashboard aggregates these into charts: today's sales, best sellers, low stock, branch comparison.

6. User & Admin Role Management

Implement **dynamic** Role-Based Access Control (RBAC): roles and permissions live in the database as data, not hardcoded in application code. This lets the Super Admin create new roles, assign fine-grained permissions, and create user logins for those roles — entirely from the admin screens, with zero developer involvement or code deploys.

6.1 What the Super Admin can do

- **Create a role** — e.g. "Store Auditor" — by giving it a name. This just adds a row to the `roles` table; it has no permissions until assigned.
- **Assign permissions to a role** — a permission-matrix screen lists every module (Billing, Inventory, Purchase, Accounting, Reports, Karigar, etc.) as rows, with View / Create / Edit / Delete / Approve as checkbox columns. Ticking a box writes a row into `role\_permissions`.
- **Create a user and attach a role** — enter name, phone/username, set a temporary password, and pick the role from a dropdown. This inserts into `users`, linked by `role\_id`.
- **Change access at any time** — editing a role's permissions instantly affects every user holding that role, with no redeploy. Deactivating a user (soft-disable) blocks their login immediately while preserving their history for audit.

6.2 Example starting roles (fully editable by Super Admin)

Starting Role	Typical Permissions
Super Admin	Full access to everything, including creating/editing roles, managing users, system settings, and financial reports. Cannot be deleted or demoted by any other role.
Admin / Owner	Full operational access; can view all branches; approves large discounts/refunds.
Branch Manager	Full access within their own branch only (enforced via `branch_id` scope on every query).
Accountant	Accounting, ledgers, vouchers, GST reports — no product/price editing.
Cashier / Sales Executive	Billing screen, customer lookup, payment collection — cannot edit product master data or view full profit margins.
Inventory Manager	Product & stock module: entries, transfers, audits, purchase entries.

These are just sensible defaults to seed the system with — since permissions are data-driven, Super Admin can rename, delete, or reshape any of them (except Super Admin itself) or add entirely new roles like "Store Auditor" or "Trainee Cashier" later.

6.3 Schema for dynamic role management

Table	Key Fields
roles	id, name, scope(all_branches/own_branch_only), is_system_role, is_active, created_by, created_at
permissions	id, module, action(view/create/edit/delete/approve) — a fixed catalogue of possible actions per module
role_permissions	role_id, permission_id — the actual grants; this is what the matrix screen edits
users	id, employee_id, username, password_hash, role_id (or user_roles if multi-role), branch_id, is_active, last_login
user_roles (optional)	user_id, role_id — only needed if a user can hold more than one role at once
activity_logs	id, user_id, action, module, ref_id (e.g. which role/permission changed), ip_address, created_at

6.4 Design decisions to lock in early

- **Branch scoping** — mark each role as ``all_branches`` or ``own_branch_only``; the backend auto-filters every query by the user's ``branch_id`` when the role is branch-scoped.
- **Single vs. multiple roles per user** — decide whether a user gets exactly one role, or a set of roles whose permissions are combined (union). A ``user_roles`` join table supports either.
- **Protecting Super Admin** — hardcode one rule outside the data-driven system: the Super Admin role can never be deleted, renamed away, or stripped of its own management permissions by anyone other than another Super Admin, so nobody can accidentally lock the business out of its own system.
- **Separate "Approve" from "Edit"** — e.g. a Cashier can raise a large-discount request, but only a role with the Approve permission on that module can authorize it — useful for discount limits, refunds, and stock write-offs.

Enforcement: check permissions in the backend on every request (never trust the frontend alone), and additionally hide/disable UI actions the user can't perform. Log every role/permission/user change to ``activity_logs``, so it's always possible to answer "who gave this role access to Reports, and when."

Security extras from the spec: bcrypt/argon2 password hashing, optional TOTP-based Two-Factor Authentication for Admin/Accountant roles, login-history tracking, and automatic session expiry after inactivity.

7. Payment Integration Workflow

For an India-based jewellery business, Razorpay or PayU are the most practical choices — both support UPI, cards, netbanking, and wallets, and both offer a hosted Checkout that avoids you having to store any card data yourself (important for PCI compliance).

1. At checkout, the backend creates an 'order' via the payment gateway's server-side API (amount, currency, invoice reference) and returns an order ID to the frontend.
2. The frontend opens the gateway's Checkout widget (Razorpay Checkout / PayU form) — the customer pays by UPI/card/netbanking directly with the gateway, so sensitive card data never touches your servers.
3. The gateway redirects/calls back with a payment ID and signature. Your backend verifies the signature using the gateway's secret key before marking the invoice paid — never trust a client-side 'success' callback alone.
4. Also configure a server-to-server webhook from the gateway (payment.captured / payment.failed) as a second, authoritative confirmation — this protects you if the customer closes the browser mid-payment.
5. On confirmed payment, write a row to ``payments``, update invoice status, and trigger the invoice PDF + SMS/WhatsApp receipt.
6. For EMI, either use the gateway's built-in EMI option (card EMI) or implement an internal Installment Schedule (``emi_plans``, ``installments``) if you're financing in-house — track due dates and send automatic reminders.
7. For cash/manual bank transfer, staff can also mark a payment as 'Cash' or 'Bank Transfer' directly without going through the gateway; both paths converge on the same ``payments`` table.

Keep API keys in environment variables/secret manager, never in source code, and always call the gateway from the backend, never expose your secret key to the browser.

8. Security, Backup & Compliance

- **Transport security:** HTTPS everywhere (Let's Encrypt via Nginx/Caddy), HSTS enabled.
- **Data at rest:** enable PostgreSQL disk encryption (cloud provider managed disks already do this) and encrypt S3 buckets storing KYC documents.
- **Backups:** automated nightly PostgreSQL dumps (`pg_dump`) to a separate storage bucket/region, plus continuous WAL archiving for point-in-time recovery; test restores quarterly.

- **Access control:** RBAC (Section 6), 2FA for admin accounts, IP allow-listing for the admin dashboard if it's only used from the shop network.
- **Audit trail:** immutable `activity\_logs` table — never allow hard deletes on financial data; use soft-delete/void flags so every change is traceable (matches the spec's 'manipulate-proof rate history' requirement).
- **GST/compliance:** keep an append-only `metal\_rates` history and `journal\_entries` — auditors and tax authorities will expect a non-editable trail.
- **Disaster recovery:** keep infrastructure-as-code (Docker Compose / Terraform) so the whole system can be redeployed on a new server within an hour if needed.

9. Deployment Plan

9.1 Where to deploy

Option	Stack	Best for
Simple/Cheapest	A single VPS (DigitalOcean Droplet, Hetzner Cloud, or AWS Lightsail), 4 vCPU / 8GB RAM to start	Runs Docker Compose: Nginx, Node API, PostgreSQL, Redis — good for one shop/a few branches
Managed/Scalable	AWS: ECS/Fargate or EC2 for the API, RDS for PostgreSQL, ElastiCache for Redis, S3 for files, CloudFront CDN for the React build	Better once you add multiple branches or need auto-scaling
Fully managed PaaS	Render / Railway / Fly.io for the API + a managed Postgres add-on	Fastest to set up, slightly higher cost per resource, less ops work

9.2 Deployment steps

1. Containerize the app: one Dockerfile for the Node.js API, one for the React build served via Nginx (or serve React statically from a CDN).
2. Write a `docker-compose.yml` (for VPS) or Terraform scripts (for AWS) defining API, PostgreSQL, Redis, and Nginx reverse proxy with TLS.
3. Set up a GitHub Actions pipeline: on push to `main` → run tests → build Docker images → push to a container registry → SSH-deploy or trigger the PaaS deploy hook.
4. Point your domain (e.g. billing.yourjewellers.com) to the server; issue a free TLS certificate via Let's Encrypt/Caddy.
5. Configure environment variables (DB credentials, payment gateway keys, SMS/WhatsApp API keys) via the host's secret manager — never commit them to git.
6. Run database migrations (`prisma migrate deploy`) as part of the deploy pipeline, not manually.
7. Set up daily automated backups (Section 8) and uptime monitoring (UptimeRobot/Better Stack) with alerts to the owner's phone/email.

9.3 Offline / poor-connectivity handling

Since a jewellery counter cannot stop billing during an internet outage, install the app as a PWA (or Electron wrapper) with a local cache of the day's product list and rates; queue invoices locally and auto-sync to the server once the connection is restored.

10. Suggested Build Roadmap

Phase	Focus	Deliverables
Phase 1 (Weeks 1-3)	Foundation	Auth/RBAC, company & branch setup, product master, customer master, basic billing (no GST edge cases yet)
Phase 2 (Weeks 4-6)	Core billing	Full GST invoicing, payment gateway integration, stock ledger, purchase entries, barcode/QR
Phase 3 (Weeks 7-9)	Operations	Gold exchange, repair job cards, order booking with rate locking, karigar management
Phase 4 (Weeks 10-12)	Accounting & Reports	Double-entry accounting, vouchers, Trial Balance/P&L;/Balance Sheet, sales/stock/GST reports, dashboard
Phase 5 (Weeks 13-14)	Compliance & Comms	TCS/PAN-Aadhaar capture, e-invoice/e-way bill (if applicable), SMS/WhatsApp integration, reminders
Phase 6 (Weeks 15-16)	Hardening & Launch	Security review, backup/DR drill, staff training, parallel-run with existing system, go-live

This is an achievable solo/small-team timeline (~4 months) if you use the AI feature list (Section 21 of the spec) as a later Phase 7 enhancement rather than a launch blocker — get core billing and accounting rock-solid first.

11. Keeping It Running Long-Term ('for lifetime')

There's no such thing as a system that runs forever with zero maintenance — but you can get very close to "set and forget" with the right habits:

- **Managed database + managed backups** so you're never manually responsible for data loss.
- **Renew the domain and TLS certs** — use auto-renewing Let's Encrypt and a domain set to auto-renew on your card.
- **Dependency updates** — schedule a quarterly pass to update Node.js/npm packages and PostgreSQL minor versions; use Dependabot to flag security patches automatically.
- **Monitoring & alerting** — uptime + error tracking (Sentry) so you hear about problems before customers do.
- **Documented runbook** — keep a short internal doc: how to restore from backup, how to reset a password, who to call if the server goes down.
- **Yearly architecture review** — as data grows (years of invoices), plan for database read replicas or archiving old years into cold storage so the live system stays fast.
- **Separate staging environment** — test new features on a staging copy before touching the live billing system, so the shop is never disrupted by a bad deploy.

With managed hosting (RDS/managed Postgres + a PaaS or a monitored VPS) and the CI/CD pipeline in Section 9, day-to-day operation requires only occasional attention rather than constant hands-on maintenance.

12. Complete End-to-End Architecture Flow

This section traces one full request lifecycle through every layer of the system — from a staff member opening the app to the data being safely backed up — and names exactly which technology is doing the work at each point. Use this as the master reference when wiring the pieces together.

12.1 Full journey — a billing transaction, step by step

#	What happens	Technology used at this point
1	Staff opens the app on the counter tablet/PC	React + TypeScript PWA (or Electron wrapper) — loaded over HTTPS from Nginx/CDN
2	Login screen submits username/password	React frontend → Node.js/NestJS API → bcrypt/argon2 verifies password hash in PostgreSQL `users` table
3	Server issues session tokens	JWT access token + refresh token; role/permissions for that user loaded once from `roles` / `role_permissions` and cached in Redis
4	Frontend requests today's products, rates, and permissions	React Query calls REST API → NestJS controllers → PostgreSQL (`products`, `metal_rates`) with Redis cache for fast repeat reads
5	Cashier scans a product barcode	USB/Bluetooth barcode scanner acts as keyboard input → React billing screen looks up SKU via API → PostgreSQL `products` + `stock_ledger`
6	Cart totals calculated live	Domain/service layer in NestJS applies making-charge, wastage %, GST % from `gst_settings` — pure business logic, no DB write yet
7	Customer looked up or quick-added	API queries/creates a row in `customers`; ledger entry prepared but not yet committed
8	Customer pays via UPI/card	Backend creates a payment gateway order (Razorpay/PayU) server-side → frontend opens hosted Checkout widget → customer pays directly with the gateway
9	Payment confirmed	Gateway sends a signed webhook + redirect callback to the backend → backend verifies signature before trusting it → written to `payments`
10	Invoice finalized	NestJS transaction commits together: `invoices`, `invoice_items`, `stock_ledger` (deduct stock), `customer_ledger`, `journal_entries` (accounting) — all in one PostgreSQL transaction so nothing is left half-done
11	PDF invoice generated	Puppeteer/pdf-lib renders the GST invoice on the backend, stores a copy in S3-compatible storage
12	Invoice delivered to customer	Background job (BullMQ on Redis) sends it via SMS/WhatsApp (MSG91/Gupshup/Twilio) and/or email, without blocking the cashier's screen
13	Dashboard updates	WebSocket/Socket.IO push (or next poll via React Query) updates live sales figures on the Admin Dashboard in real time
14	Every action logged	Middleware in NestJS writes an entry to `activity_logs` (who, what, when) for every create/edit/delete across all of the above
15	Nightly automated jobs run	Scheduled workers: gold/silver rate refresh from a live rate API, low-stock alerts, PostgreSQL backup (pg_dump + WAL archiving) to a separate storage region

#	What happens	Technology used at this point
16	Monitoring watches it all	Sentry captures errors, Prometheus/Grafana or UptimeRobot tracks uptime and response times, alerting the owner/dev if anything breaks

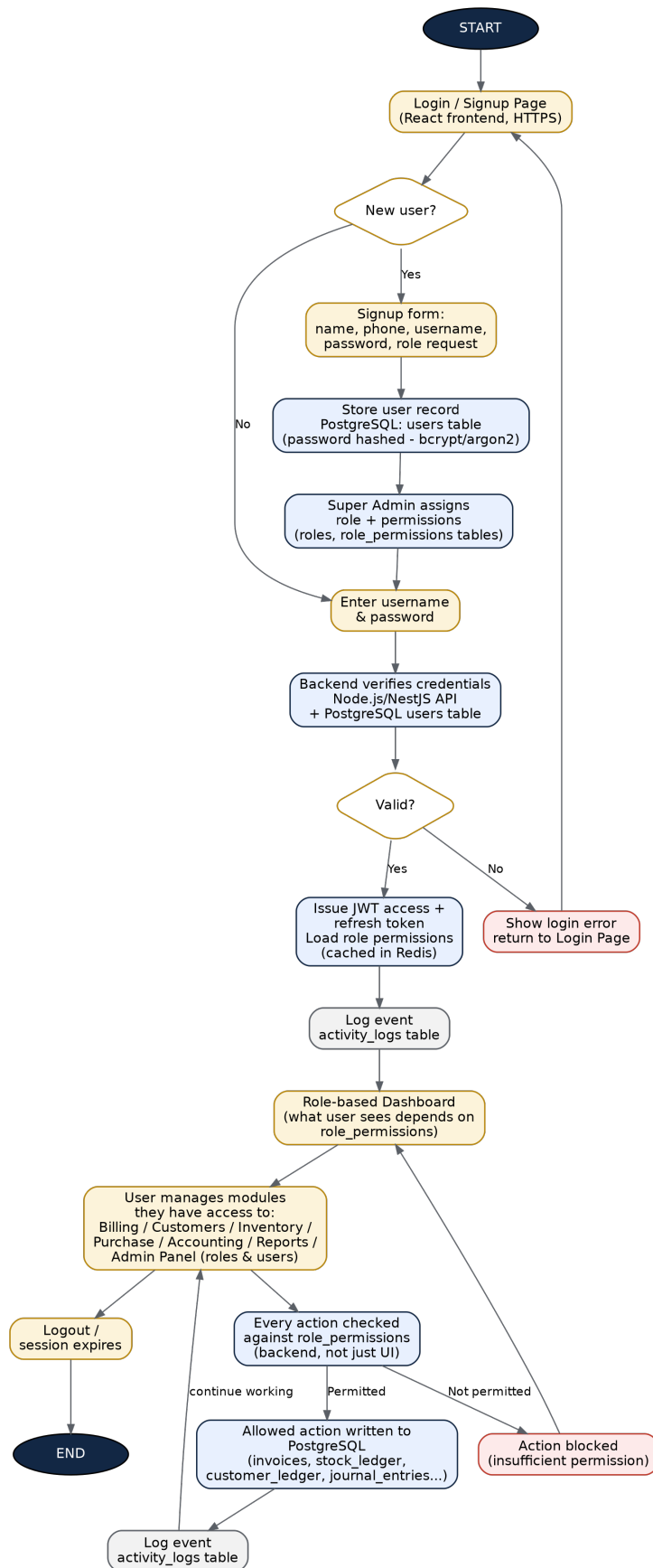
12.2 Layer-by-layer map (quick lookup)

Layer	Technology	Used for
Presentation	React + TypeScript, TailwindCSS, shadcn/ui, PWA/Electron	Billing screen, admin dashboard, all forms and reports the user sees
API / Application	Node.js + NestJS (REST), JWT auth, RBAC middleware	Every request from the frontend passes through here first — auth, validation, permission checks
Domain / Business Logic	TypeScript service classes inside NestJS	GST calculation, tunch/purity conversion, gold-rate locking, wastage & melting-loss math
Data — primary	PostgreSQL 16 via Prisma/TypeORM	Customers, products, invoices, accounting, roles/permissions — the single source of truth
Data — cache/queue	Redis + BullMQ	Session/permission cache, background jobs: SMS/WhatsApp sending, reminders, nightly rate refresh, backups
Data — files	S3-compatible object storage (AWS S3 / Cloudflare R2 / MinIO)	Product images, KYC documents, hallmark certificates, generated invoice PDFs
External integrations	Razorpay/PayU, MSG91/Gupshup/Twilio, live gold-rate API, GSP for e-invoicing	Payments, customer notifications, daily metal rates, GST e-invoice/e-way bill
Infrastructure	Docker containers, Nginx reverse proxy, GitHub Actions CI/CD, VPS/AWS/PaaS hosting	How the whole stack above is built, deployed, and kept running (Section 9)
Observability & Safety	Sentry, Prometheus/Grafana or UptimeRobot, automated pg_dump + WAL backups	Watches for errors/downtime and protects against data loss (Section 8 & 11)

How to read this: Section 12.1 follows one real transaction top to bottom in time order; Section 12.2 is the same system sliced horizontally by layer. Together they answer both "what happens when a sale is made" and "what technology am I touching when I work on module X" — the two questions that come up most often once development starts.

13. Login-to-Usage Flow Diagram

A visual walkthrough of the exact journey requested: starting at Login/Signup, showing where the user is stored, how a role gets attached, and how the user then manages the system from their role-based dashboard — with every box naming the technology/table involved.



Gold boxes = user-facing screens · Blue boxes = backend/database actions · Grey boxes = audit logging · Red boxes = error/blocked paths · Diamonds = decision points.